

Exercise 3

Deadline: 11.06.2026, 21:00

In this exercise we introduce the pytorch framework, a leading open-source Python library for neural network research, mainly developed by FacebookAI. It supports both CPU- and GPU-based execution. Neural networks (or any other computation) are expressed in terms of computation graphs, which define functional relationships between variables (e.g. Tensors) and allow to calculate the gradients of any nested expression automatically, from within Python. pytorch tutorials and documentation can be found at <http://pytorch.org>.

Regulations

Please create a Jupyter notebook `cnn.ipynb` for your solution and export it into `cnn.html`. Zip both files into a single archive. Zip all files into a single archive `ex03.zip` and upload this file to MaMPF before the given deadline.

Moreover, please set your **Anzeigenname/display name** and **Name in Uebungsgruppen/name in tutorials** in MaMPF to your real name, which should be identical to your name in `muesli` and make sure you **join the submission** of your team via the invitation code before the submission deadline. Check out <https://mampf.blog/handing-in-homework-assignments> for instructions.

1 Introduction (5 Points)

First you need to make yourself familiar with pytorch. The following code (available as external link on MaMPF and under <https://tinyurl.com/HD-AML-intro-py>) defines a simple neural network with 2 hidden fully-connected layers.

```
1
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 import torch
6 import torch.optim as optim
7 from torch.utils.data import DataLoader
8
9 import torchvision.datasets as datasets
10 import torchvision.transforms as transforms
11
12 from torch.nn.functional import conv2d, max_pool2d, cross_entropy
13
14 plt.rc("figure", dpi=100)
15
16 batch_size = 100
17
18 # transform images into normalized tensors
19 transform = transforms.Compose([
20     transforms.ToTensor(),
21     transforms.Normalize(mean=(0.5,), std=(0.5,))
22 ])
23
24 train_dataset = datasets.MNIST(
25     "./",
26     download=True,
27     train=True,
28     transform=transform,
29 )
30
31 test_dataset = datasets.MNIST(
32     "./",
33     download=True,
```

```
34     train=False,
35     transform=transform,
36 )
37
38 train_dataloader = DataLoader(
39     dataset=train_dataset,
40     batch_size=batch_size,
41     shuffle=True,
42     num_workers=1,
43     pin_memory=True,
44 )
45
46 test_dataloader = DataLoader(
47     dataset=test_dataset,
48     batch_size=batch_size,
49     shuffle=False,
50     num_workers=1,
51     pin_memory=True,
52 )
53
54 def init_weights(shape):
55     # Kaiming He initialization (a good initialization is important)
56     # https://arxiv.org/abs/1502.01852
57     std = np.sqrt(2. / shape[0])
58     w = torch.randn(size=shape) * std
59     w.requires_grad = True
60     return w
61
62
63 def rectify(x):
64     # Rectified Linear Unit (ReLU)
65     return torch.max(torch.zeros_like(x), x)
66
67
68 class RMSprop(optim.Optimizer):
69     """
70     This is a reduced version of the PyTorch internal RMSprop optimizer
71     It serves here as an example
72     """
73     def __init__(self, params, lr=1e-3, alpha=0.5, eps=1e-8):
74         defaults = dict(lr=lr, alpha=alpha, eps=eps)
75         super(RMSprop, self).__init__(params, defaults)
76
77     def step(self):
78         for group in self.param_groups:
79             for p in group['params']:
80                 grad = p.grad.data
81                 state = self.state[p]
82
83                 # state initialization
84                 if len(state) == 0:
85                     state['square_avg'] = torch.zeros_like(p.data)
86
87                 square_avg = state['square_avg']
88                 alpha = group['alpha']
89
90                 # update running averages
91                 square_avg.mul_(alpha).addcmul_(grad, grad, value=1 - alpha)
92                 avg = square_avg.sqrt().add_(group['eps'])
93
94                 # gradient update
95                 p.data.addcddiv_(grad, avg, value=-group['lr'])
96
97
98 # define the neural network
99 def model(x, w_h, w_h2, w_o):
100     h = rectify(x @ w_h)
101     h2 = rectify(h @ w_h2)
```

```
102     pre_softmax = h2 @ w_o
103     return pre_softmax
104
105
106     # initialize weights
107
108     # input shape is (B, 784)
109     w_h = init_weights((784, 625))
110     # hidden layer with 625 neurons
111     w_h2 = init_weights((625, 625))
112     # hidden layer with 625 neurons
113     w_o = init_weights((625, 10))
114     # output shape is (B, 10)
115
116     optimizer = RMSprop(params=[w_h, w_h2, w_o])
117
118
119     n_epochs = 100
120
121     train_loss = []
122     test_loss = []
123
124     # put this into a training loop over 100 epochs
125     for epoch in range(n_epochs + 1):
126         train_loss_this_epoch = []
127         for idx, batch in enumerate(train_dataloader):
128             x, y = batch
129
130             # our model requires flattened input
131             x = x.reshape(batch_size, 784)
132             # feed input through model
133             noise_py_x = model(x, w_h, w_h2, w_o)
134
135             # reset the gradient
136             optimizer.zero_grad()
137
138             # the cross-entropy loss function already contains the softmax
139             loss = cross_entropy(noise_py_x, y, reduction="mean")
140
141             train_loss_this_epoch.append(float(loss))
142
143             # compute the gradient
144             loss.backward()
145             # update weights
146             optimizer.step()
147
148         train_loss.append(np.mean(train_loss_this_epoch))
149
150     # test periodically
151     if epoch % 10 == 0:
152         print(f"Epoch: {epoch}")
153         print(f"Mean Train Loss: {train_loss[-1]:.2e}")
154         test_loss_this_epoch = []
155
156         # no need to compute gradients for validation
157         with torch.no_grad():
158             for idx, batch in enumerate(test_dataloader):
159                 x, y = batch
160                 x = x.reshape(batch_size, 784)
161                 noise_py_x = model(x, w_h, w_h2, w_o)
162
163                 loss = cross_entropy(noise_py_x, y, reduction="mean")
164                 test_loss_this_epoch.append(float(loss))
165
166         test_loss.append(np.mean(test_loss_this_epoch))
167
168     print(f"Mean Test Loss: {test_loss[-1]:.2e}")
169
```

```
170 plt.plot(np.arange(n_epochs + 1), train_loss, label="Train")
171 plt.plot(np.arange(1, n_epochs + 2, 10), test_loss, label="Test")
172 plt.title("Train and Test Loss over Training")
173 plt.xlabel("Epoch")
174 plt.ylabel("Loss")
175 plt.legend()
```

Task: Install pytorch (best with conda), convert `intro.py` into a Jupyter notebook and run the code.

2 Dropout (8 Points)

We want to use dropout learning for our network. Therefore, implement the function

```
def dropout(X, p_drop=0.5):
    ...
```

that sets random elements of X to zero (do *not* use pytorch's existing dropout functionality).

Dropout:

- **If** $0 < p_{\text{drop}} < 1$:
For every element $x_i \in X$ draw Φ_i randomly from a binomial distribution with $p = p_{\text{drop}}$.
Then reassign

$$x_i \rightarrow \begin{cases} 0 & \text{if } \Phi = 1 \\ \frac{x_i}{1-p_{\text{drop}}} & \text{if } \Phi = 0 \end{cases}$$

- **Else:**
Return the unchanged X .

You can now enable the dropout functionality. To this end, implement a new model

```
def dropout_model(X, w_h, w_h2, w_o, p_drop_input, p_drop_hidden):
    ...
```

containing the same fully-connected layers as in function `model()` of task 1, but now with three dropout steps. Dropout is applied to the *input* of each layer.

Task: Explain in a few sentences how the dropout method works and how it reduces overfitting. Train the model using dropout and report train and test errors. Why do we need a different model configuration for evaluating the test loss? Compare the test error with the test error from Section 1.

3 Parametric Relu (10 Points)

Instead of a simple rectify mapping (aka rectified linear unit; Relu) we want to add a parametric Relu that maps every element x_i of the input X to

$$x_i \rightarrow \begin{cases} x_i & x_i > 0 \\ a_i x_i & x_i \leq 0 \end{cases}.$$

A detailed description can be found in the paper **Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification** (see <http://arxiv.org/abs/1502.01852>). The crux of this method are the learnable weights a that need to be adjusted during training. Define the function

```
def PRelu(X, a):
    ...
```

that creates a PRelu layer by mapping $X \rightarrow \text{PRelu}(X)$.

Incorporate the parameters a into the **params** list and make sure that it is optimized during training.

Task: Compare the results with the previous models.

4 Convolutional layers (17 Points)

In this exercise we want to create a similar neural network to LeNet from Yann LeCun. LeNet was designed for handwritten and machine-printed character recognition. It relies on convolutional layers that transform the input image by convolution with multiple learnable filters. LeNet contains convolutional layers paired with sub-sampling layers as displayed in Figure 1. The Subsampling is done via max pooling which reduces an area of the image to one pixel with the maximum value of the area. Both functions are already available in pytorch:

```
from torch.nn.functional import conv2d, max_pool2d

convolutional_layer = rectify(conv2d(previous_layer, weightvector))
# reduces (2,2) window to 1 pixel
subsampling_layer = max_pool2d(convolutional_layer, (2, 2))
out_layer = dropout(subsampling_layer, p_drop_input)
```

4.1 Create a Convolutional network

Now we can design our own convolutional neural network that classifies the handwritten numbers from MNIST.

Implementation task:

- Make sure that the input image has the correct shape:

```
trainX = trainX.reshape(-1, 1, 28, 28) #training data
testX = testX.reshape(-1, 1, 28, 28) #test data
```

- Replace the first hidden layer **h** with 3 convolutional layers (including subsampling and dropout)
- Connect the convolutional layers to the vectorized layer **h2** by flattening the input with **torch.reshape**.
- The shape of the weight parameter for **conv2d** determines the number of filters f , the number of input images pic_{in} , and the kernel size $k = (k_x, k_y)$. You can initialize the weights with

```
init_weights((f, pic_in, k_x, k_y))
```

Make a neural network with

convolutional layer:	first	second	third
f	32	64	128
pic_{in}	1	32	64
k_x	5	5	3
k_y	5	5	3

and add the weight vectors to the **params** list.

- In Section 4.2 you will determine the number of output pixels of the CNN. Use it to adjust the size of the rectifier layer to

```
w_h2 = init_weights((number_of_output_pixel, 625))
```

- Use a pre-softmax output layer with 625 inputs and 10 outputs (as before).

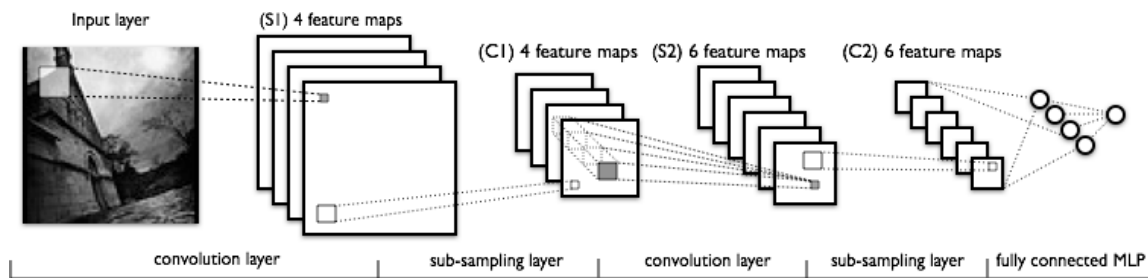


Abbildung 1: Sketch of convolutional neural network similar to LeNet

4.2 Application of Convolutional network

Task:

- Draw a sketch of the network (like Figure 1) and note the sizes of the filter images (This will help you to determine how many pixels there are in the last convolution layer).
- Train the model. Then, plot:
 - one image from the test set
 - its convolution with 3 filters of the first convolutional layer
 - the corresponding filter weights (these should be 5 by 5 images).

Finally, choose **one** of the following tasks:

- add or remove one convolutional layer (you may adjust the number of filters)
- increase the filter size (you may plot some pictures i
- apply a random linear shift to the trainings images. Does this reduce overfitting?
- use unisotropic filters $k_x \neq k_y$
- create a network architecture of your choice and see if you can improve on the previous results

and compare the new test error.

Ideally you should create an overview table that lists the test errors from all sections.